

Week 7 Skill

To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs

```
divya@Divyasree:~$ cd
divya@Divyasree:~$ cd input_redirect_project
-bash: cd: input_redirect_project: No such file or directory
divya@Divyasree:~$ mkdir input_redirect_project
divya@Divyasree:~$ cd input_redirect_project
divya@Divyasree:~/input_redirect_project$ pwd
/home/divya/input_redirect_project
divya@Divyasree:~/input_redirect_project$ nano input.txt
divya@Divyasree:~/input_redirect_project$ cat input.txt
Linux File I/O
Input Redirection
System Calls
Ubuntu Operating System
divya@Divyasree:~/input_redirect_project$ nano input_redirect.c
divya@Divyasree:~/input_redirect_project$ gcc input_redirect.c -o input_redirect
divya@Divyasree:~/input_redirect_project$ ls
input.txt  input_redirect  input_redirect.c
divya@Divyasree:~/input_redirect_project$ ./input_redirect
===== Input Redirection Manager =====

Enter command using input redirection:
Example: wc -l < input.txt
> wc -l < input.txt

Parsed Command: wc -l
Input File: input.txt
Input file opened successfully: input.txt
File descriptor: 3
Standard input redirected from input.txt
4

Standard input restored successfully.

Parsed Command: wc -l
Input File: input.txt
Input file opened successfully: input.txt
File descriptor: 3
Standard input redirected from input.txt
4

Standard input restored successfully.

===== Input Redirection Test Completed =====
divya@Divyasree:~/input_redirect_project$ wc -l
^C
divya@Divyasree:~/input_redirect_project$ ./input_redirect
===== Input Redirection Manager =====

Enter command using input redirection:
Example: wc -l < input.txt
> wc -l < missing.txt

Parsed Command: wc -l
Input File: missing.txt
Error opening input file: No such file or directory

===== Input Redirection Test Completed =====
divya@Divyasree:~/input_redirect_project$ ./input_redirect
===== Input Redirection Manager =====

Enter command using input redirection:
Example: wc -l < input.txt
> |
```

Txt: `anj@bjusree: /input.txt`

```
GNU nano 7.2
Linux File I/O
Input Redirection
System Calls
Ubuntu Operating System
```

Code:

```
GNU nano 7.2                                     input_redirect.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 200

/*
 * Save original stdin so that it can be restored later.
 */
int saved_stdin;

/*
 * Parse input redirection syntax.
 * Example:
 *     wc -l < input.txt
 *
 * The function separates:
 *     command = wc -l
 *     filename = input.txt
 */
int parse_redirection(char *input, char *command, char *filename)
{
    char *redirect = strchr(input, '<');

    if (redirect == NULL)
    {
```

```

    *redirect = '\\0';

    strcpy(command, input);

    redirect++;

    while (*redirect == ' ')
        redirect++;

    strcpy(filename, redirect);

    /* Remove trailing spaces */
    filename[strlen(filename)] = '\\0';

    int len = strlen(filename);

    while (len > 0 && filename[len - 1] == ' ')
    {
        filename[len - 1] = '\\0';
        len--;
    }

    return 1;
}

/*
Restore original stdin.
*/

```

```

GNU nano 7.2      input_redirect.c
{
    perror("dup2 restore");
}

printf("\\nStandard input restored successfully.\\n");
}

/*
Execute command with input redirection.
*/
void execute_command(char *command, char *filename)
{
    /*
    Open input file.
    */
    int fd = open(filename, O_RDONLY);

    if (fd == -1)
    {
        perror("Error opening input file");
        return;
    }

    printf("Input file opened successfully: %s\\n", filename);
    printf("File descriptor: %d\\n", fd);

    /*
    Save original stdin.
    */
}

```

```

if (saved_stdin == -1)
{
    perror("dup");
    close(fd);
    return;
}

/*
    Redirect stdin to the file.
*/
if (dup2(fd, STDIN_FILENO) == -1)
{
    perror("dup2");
    close(fd);
    close(saved_stdin);
    return;
}

printf("Standard input redirected from %s\n", filename);

close(fd);

/*
    Create child process.
*/
pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
}

if (pid == 0)
{
    /*
        Child executes the command.
    */

    char command_copy[MAX_INPUT];

    strcpy(command_copy, command);

    char *args[20];
    int count = 0;

    char *token = strtok(command_copy, " ");

    while (token != NULL && count < 19)
    {
        args[count++] = token;
        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    execvp(args[0], args);
}

```

```

        close(saved_stdin);
    }

int main()
{
    char input[MAX_INPUT];
    char command[MAX_INPUT];
    char filename[MAX_INPUT];

    printf("==== Input Redirection Manager ==== \n");

    printf("\nEnter command using input redirection: \n");
    printf("Example: wc -l < input.txt \n");
    printf("> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    /*
     * Parse < symbol.
     */
    if (!parse_redirection(input, command, filename))
    {
        printf("Error: Input redirection '<' not found. \n");
        return 1;
    }

    return 1;

    printf("\nParsed Command: %s \n", command);
    printf("Input File: %s \n", filename);

    execute_command(command, filename);

    printf("\n==== Input Redirection Test Completed ==== \n");

    return 0;
}

```

TO parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection

```

divya@Divyasree:~$ cd
divya@Divyasree:~$ cd output_redirect_project
divya@Divyasree:~/output_redirect_project$ pwd
/home/divya/output_redirect_project
divya@Divyasree:~/output_redirect_project$ nano output_redirect.c
divya@Divyasree:~/output_redirect_project$ gcc output_redirect.c -o output_redirect
divya@Divyasree:~/output_redirect_project$ ls
location.txt  output_redirect  test.txt
output.txt   output_redirect.c
divya@Divyasree:~/output_redirect_project$ ./output_redirect
==== Output Redirection Manager ====

Enter command using output redirection:
Example: ls -l > output.txt
> ls -l > output.txt

Parsed Command: ls -l
Output File: output.txt
Output file opened successfully: output.txt
File descriptor: 3

Standard output restored successfully.

==== Output Redirection Test Completed ====
divya@Divyasree:~/output_redirect_project$ ls
location.txt  output_redirect  test.txt
output.txt   output_redirect.c
divya@Divyasree:~/output_redirect_project$ cat output.txt
total 36
-rw-r--r-- 1 divya divya   36 Sep 27 10:58 location.txt
-rw-r--r-- 1 divya divya    0 Sep 27 11:23 output.txt
-rwxr-xr-x 1 divya divya 16944 Sep 27 11:22 output_redirect
-rw-r--r-- 1 divya divya 4306 Sep 27 11:15 output_redirect.c

```

```

total 36
-rw-r--r-- 1 divya divya 36 Sep 27 10:58 location.txt
-rw-r--r-- 1 divya divya 0 Sep 27 11:23 output.txt
-rwxr-xr-x 1 divya divya 16944 Sep 27 11:22 output_redirect
-rw-r--r-- 1 divya divya 4306 Sep 27 11:15 output_redirect.c
-rw-r--r-- 1 divya divya 7 Sep 27 11:00 test.txt
divya@Divyasree:~/output_redirect_project$ ./output_redirect
===== Output Redirection Manager =====

Enter command using output redirection:
Example: ls -l > output.txt
> pwd > location.txt

Parsed Command: pwd
Output File: location.txt
Output file opened successfully: location.txt
File descriptor: 3

Standard output restored successfully.

===== Output Redirection Test Completed =====
divya@Divyasree:~/output_redirect_project$ cat location.txt
/home/divya/output_redirect_project
divya@Divyasree:~/output_redirect_project$ ./output_redirect
===== Output Redirection Manager =====

Enter command using output redirection:
Example: ls -l > output.txt
> echo First > test.txt

Parsed Command: echo First
Output File: test.txt
Output file opened successfully: test.txt

===== Output Redirection Test Completed =====
divya@Divyasree:~/output_redirect_project$ cat test.txt
First
divya@Divyasree:~/output_redirect_project$ ./output_redirect
===== Output Redirection Manager =====

Enter command using output redirection:
Example: ls -l > output.txt
> echo Second > test.txt

Parsed Command: echo Second
Output File: test.txt
Output file opened successfully: test.txt
File descriptor: 3

Standard output restored successfully.

===== Output Redirection Test Completed =====
divya@Divyasree:~/output_redirect_project$ cat test.txt
Second
divya@Divyasree:~/output_redirect_project$ ./output_redirect
===== Output Redirection Manager =====

Enter command using output redirection:
Example: ls -l > output.txt
> ls -l
Error: Output redirection '>' not found.
divya@Divyasree:~/output_redirect_project$ ./output_redirect
===== Output Redirection Manager =====

Enter command using output redirection:
Example: ls -l > output.txt
> echo Hello > /root/test.txt

```

```
divya@Divyasree:~/output_redirect_project$ ./output_redirect
===== Output Redirection Manager =====

Enter command using output redirection:
Example: ls -l > output.txt
> ls -l
Error: Output redirection '>' not found.
divya@Divyasree:~/output_redirect_project$ ./output_redirect
===== Output Redirection Manager =====

Enter command using output redirection:
Example: ls -l > output.txt
> echo Hello > /root/test.txt

Parsed Command: echo Hello
Output File: /root/test.txt
Error opening output file: Permission denied

===== Output Redirection Test Completed =====
divya@Divyasree:~/output_redirect_project$ |
```

Code:

```
GNU nano 7.2 output_redirect.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 200

int saved_stdout;

/*
    Parse output redirection.

    Example:
        ls -l > output.txt

    Separates:
        command = ls -l
        filename = output.txt
*/
int parse_redirection(char *input, char *command, char *filename)
{
    char *redirect = strchr(input, '>');

    if (redirect == NULL)
    {
        return 0;
    }
}
```

```
while (*redirect == ' ')
    redirect++;

strcpy(filename, redirect);

filename[strlen(filename)] = '\0';

/* Remove trailing spaces */
int len = strlen(filename);

while (len > 0 && filename[len - 1] == ' ')
{
    filename[len - 1] = '\0';
    len--;
}

/* Remove leading spaces from command */
char *start = command;

while (*start == ' ')
    start++;

if (start != command)
    memmove(command, start, strlen(start) + 1);

/* Remove trailing spaces from command */
len = strlen(command);
```

```

while (*redirect == ' ')
    redirect++;

strcpy(filename, redirect);

filename[strcspn(filename, "\n")] = '\0';

/* Remove trailing spaces */
int len = strlen(filename);

while (len > 0 && filename[len - 1] == ' ')
{
    filename[len - 1] = '\0';
    len--;
}

/* Remove leading spaces from command */
char *start = command;

while (*start == ' ')
    start++;

if (start != command)
    memmove(command, start, strlen(start) + 1);

/* Remove trailing spaces from command */
len = strlen(command);

while (len > 0 && command[len - 1] == ' ')
{
    command[len - 1] = '\0';
    len--;
}

return 1;
}

/*
Restore original stdout.
*/
void restore_stdout()
{
    if (dup2(saved_stdout, STDOUT_FILENO) == -1)
    {
        perror("dup2 restore");
    }

    printf("\nStandard output restored successfully.\n");
}

/*
Execute command with output redirection.
*/
void execute_command(char *command, char *filename)
{
    /*
    Open/create output file.

    O_WRONLY -> write only
    O_CREAT -> create if it doesn't exist
    O_TRUNC -> clear old contents

```

GNU nano 7.2

```
O_WRONLY | O_CREAT | O_TRUNC,  
0644  
);  
  
if (fd == -1)  
{  
    perror("Error opening output file");  
    return;  
}  
  
printf("Output file opened successfully: %s\n", filename);  
printf("File descriptor: %d\n", fd);  
  
/*  
    Save original stdout.  
*/  
saved_stdout = dup(STDOUT_FILENO);  
  
if (saved_stdout == -1)  
{  
    perror("dup");  
    close(fd);  
    return;  
}  
  
/*  
    Redirect stdout to the file.  
*/  
if (dup2(fd, STDOUT_FILENO) == -1)  
{  
    perror("dup2");  
    close(fd);  
    close(saved_stdout);  
    return;  
}  
  
close(fd);  
  
/*  
    Create child process.  
*/  
pid_t pid = fork();  
  
if (pid < 0)  
{  
    |
```

```

        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}

/*
 * Parent waits for child.
 */
waitpid(pid, NULL, 0);

/*
 * Restore original stdout.
 */
restore_stdout();

close(saved_stdout);
}

int main()
{
    char input[MAX_INPUT];
    char command[MAX_INPUT];
    char filename[MAX_INPUT];

    printf("==== Output Redirection Manager ====\n");

    printf("\nEnter command using output redirection:\n");
    printf("Example: ls -l > output.txt\n");
    printf("> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    /*
     * Parse > symbol.
     */
    if (!parse_redirection(input, command, filename))
    {
        printf("Error: Output redirection '>' not found.\n");
        return 1;
    }

    if (strlen(command) == 0 || strlen(filename) == 0)
    {
        printf("Error: Invalid output redirection syntax.\n");
        return 1;
    }

    printf("\nParsed Command: %s\n", command);
    printf("Output File: %s\n", filename);

    execute_command(command, filename);

    printf("\n==== Output Redirection Test Completed ====\n");

    return 0;
}

```

To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios

```
divya@Divyasree:~$ cd
divya@Divyasree:~$ mkdir append_redirect_project
divya@Divyasree:~$ cd append_redirect_project
divya@Divyasree:~/append_redirect_project$ pwd
/home/divya/append_redirect_project
divya@Divyasree:~/append_redirect_project$ nano append_redirect.c
divya@Divyasree:~/append_redirect_project$ gcc append_redirect.c -o append_redirect
divya@Divyasree:~/append_redirect_project$ ls
append_redirect  append_redirect.c
divya@Divyasree:~/append_redirect_project$ ./append_redirect
===== Append Redirection Manager =====

Enter command using append redirection:
Example: echo Hello >> output.txt
> echo First >> output.txt

Parsed Command: echo First
Output File: output.txt
Output file opened in APPEND mode.
File: output.txt
File descriptor: 3

Standard output restored successfully.

===== Append Redirection Test Completed =====
divya@Divyasree:~/append_redirect_project$ cat output.txt
First
divya@Divyasree:~/append_redirect_project$ ./append_redirect
===== Append Redirection Manager =====

Enter command using append redirection:
Example: echo Hello >> output.txt
> echo First >> output.txt

Parsed Command: echo First
Output File: output.txt
Output file opened in APPEND mode.
File: output.txt
File descriptor: 3

Standard output restored successfully.

===== Append Redirection Test Completed =====
divya@Divyasree:~/append_redirect_project$ cat output.txt
First
Second
divya@Divyasree:~/append_redirect_project$ ./append_redirect
===== Append Redirection Manager =====

Enter command using append redirection:
Example: echo Hello >> output.txt
> echo Third >> output.txt

Parsed Command: echo Third
Output File: output.txt
Output file opened in APPEND mode.
File: output.txt
File descriptor: 3

Standard output restored successfully.

===== Append Redirection Test Completed =====
divya@Divyasree:~/append_redirect_project$ cat output.txt
First
Second
Third
divya@Divyasree:~/append_redirect_project$ ./append_redirect
===== Append Redirection Manager =====
```

```

Enter command using append redirection:
Example: echo Hello >> output.txt
> pwd >> output.txt

Parsed Command: pwd
Output File: output.txt
Output file opened in APPEND mode.
File: output.txt
File descriptor: 3

Standard output restored successfully.

===== Append Redirection Test Completed =====
divya@Divyasree:~/append_redirect_project$ cat output.txt
First
Second
Third
/home/divya/append_redirect_project
divya@Divyasree:~/append_redirect_project$ ls -l output.txt
-rw-r--r-- 1 divya divya 55 Sep 27 14:26 output.txt
divya@Divyasree:~/append_redirect_project$ wc -l output.txt
4 output.txt
divya@Divyasree:~/append_redirect_project$ ^C
divya@Divyasree:~/append_redirect_project$ rm newfile.txt
rm: cannot remove 'newfile.txt': No such file or directory
divya@Divyasree:~/append_redirect_project$ ./append_redirect
===== Append Redirection Manager =====

Enter command using append redirection:
Example: echo Hello >> output.txt
> echo Created >> newfile.txt



Enter command using append redirection:
Example: echo Hello >> output.txt
> echo Hello > output.txt
Error: Append operator '>>' not found.
divya@Divyasree:~/append_redirect_project$ |


```

Code:

```
GNU nano 7.2                                     append_redirect.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 200

int saved_stdout;

/* Parse command >> filename */
int parse_append(char *input, char *command, char *filename)
{
    char *redirect = strstr(input, ">>");

    if (redirect == NULL)
    {
        return 0;
    }

    *redirect = '\0';

    strcpy(command, input);

    redirect += 2;

    while (*redirect == ' ')
        redirect++;

    /* Remove trailing spaces from filename */
    int len = strlen(filename);

    while (len > 0 && filename[len - 1] == ' ')
    {
        filename[len - 1] = '\0';
        len--;
    }

    /* Remove leading spaces from command */
    char *start = command;

    while (*start == ' ')
        start++;

    if (start != command)
        memmove(command, start, strlen(start) + 1);

    /* Remove trailing spaces from command */
    len = strlen(command);

    while (len > 0 && command[len - 1] == ' ')
    {
        command[len - 1] = '\0';
        len--;
    }

    return 1;
}
```

```

GNU nano 7.2                                     append_redirect.c
    {
        perror("dup2 restore");
    }

    printf("\nStandard output restored successfully.\n");
}

/* Execute command with append redirection */
void execute_command(char *command, char *filename)
{
    /*
        O_WRONLY -> write mode
        O_CREAT  -> create file if it doesn't exist
        O_APPEND -> add data at the end
    */
    int fd = open(
        filename,
        O_WRONLY | O_CREAT | O_APPEND,
        0644
    );

    if (fd == -1)
    {
        perror("Error opening output file");
        return;
    }

    printf("Output file opened in APPEND mode.\n");
    printf("File: %s\n", filename);
}

GNU nano 7.2                                     append_redirect.c
    saved_stdout = dup(STDOUT_FILENO);

    if (saved_stdout == -1)
    {
        perror("dup");
        close(fd);
        return;
    }

    /* Redirect stdout to file */
    if (dup2(fd, STDOUT_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        close(saved_stdout);
        return;
    }

    close(fd);

    /* Create child process */
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");

        dup2(saved_stdout, STDOUT_FILENO);
        close(saved_stdout);
    }
}

```

```

}

if (pid == 0)
{
    char command_copy[MAX_INPUT];

    strcpy(command_copy, command);

    char *args[20];
    int count = 0;

    char *token = strtok(command_copy, " ");

    while (token != NULL && count < 19)
    {
        args[count++] = token;
        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}

/* Parent waits */
waitpid(pid, NULL, 0);

```

```

One nano 1.12
append_redirection.c

char input[MAX_INPUT];
char command[MAX_INPUT];
char filename[MAX_INPUT];

printf("==== Append Redirection Manager =====\n");

printf("\nEnter command using append redirection:\n");
printf("Example: echo Hello >> output.txt\n");
printf("> ");

if (fgets(input, sizeof(input), stdin) == NULL)
    return 1;

input[strcspn(input, "\n")] = '\0';

/* Parse >> */
if (!parse_append(input, command, filename))
{
    printf("Error: Append operator '>>' not found.\n");
    return 1;
}

if (strlen(command) == 0 || strlen(filename) == 0)
{
    printf("Error: Invalid append syntax.\n");
    return 1;
}

printf("\nParsed Command: %s\n", command);

```

```

    if (strlen(command) == 0 || strlen(filename) == 0)
    {
        printf("Error: Invalid append syntax.\n");
        return 1;
    }

    printf("\nParsed Command: %s\n", command);
    printf("Output File: %s\n", filename);

    execute_command(command, filename);

    printf("\n==== Append Redirection Test Completed =====\n");

    return 0;
}

```

TO parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

```

divya@Divyasree:~$ cd
divya@Divyasree:~$ mkdir stderr_redirect_project
divya@Divyasree:~$ cd stderr_redirect_project
divya@Divyasree:~/stderr_redirect_project$ pwd
/home/divya/stderr_redirect_project
divya@Divyasree:~/stderr_redirect_project$ nano stderr_redirect.c
divya@Divyasree:~/stderr_redirect_project$ gcc stderr_redirect.c -o stderr_redirect
divya@Divyasree:~/stderr_redirect_project$ ls
stderr_redirect  stderr_redirect.c
divya@Divyasree:~/stderr_redirect_project$ ./stderr_redirect
===== STDERR Redirection Manager =====

Enter command using stderr redirection:
Example: ls /missing 2> errors.txt
> ls /does_not_exist 2> errors.txt

Parsed Command: ls /does_not_exist
Error Output File: errors.txt
Error output file opened successfully: errors.txt
File descriptor: 3
Standard error restored successfully.
Child process exited with status: 2

===== STDERR Redirection Test Completed =====
divya@Divyasree:~/stderr_redirect_project$ cat errors.txt
ls: cannot access '/does_not_exist': No such file or directory
divya@Divyasree:~/stderr_redirect_project$ ./stderr_redirect
===== STDERR Redirection Manager =====

Enter command using stderr redirection:
Example: ls /missing 2> errors.txt
> cat missing_file.txt 2> errors2.txt

```

```

Parsed Command: cat missing_file.txt
Error Output File: errors2.txt
Error output file opened successfully: errors2.txt
File descriptor: 3
Standard error restored successfully.
Child process exited with status: 1

===== STDERR Redirection Test Completed =====
divya@Divyasree:~/stderr_redirect_project$ cat errors2.txt
cat: missing_file.txt: No such file or directory
divya@Divyasree:~/stderr_redirect_project$ ^C
divya@Divyasree:~/stderr_redirect_project$ ./stderr_redirect
===== STDERR Redirection Manager =====

Enter command using stderr redirection:
Example: ls /missing 2> errors.txt
> wrongcommand 2> command_error.txt

Parsed Command: wrongcommand
Error Output File: command_error.txt
Error output file opened successfully: command_error.txt
File descriptor: 3
Standard error restored successfully.
Child process exited with status: 1

===== STDERR Redirection Test Completed =====
divya@Divyasree:~/stderr_redirect_project$ cat command_error.txt
execvp: No such file or directory
divya@Divyasree:~/stderr_redirect_project$ ./stderr_redirect
===== STDERR Redirection Manager =====

Enter command using stderr redirection:
Example: ls /missing 2> errors.txt
> ls 2> no_error.txt

Parsed Command: ls
Error Output File: no_error.txt
Error output file opened successfully: no_error.txt
File descriptor: 3
command_error.txt  errors2.txt  stderr_redirect
errors.txt        no_error.txt  stderr_redirect.c
Standard error restored successfully.
Child process exited with status: 0

===== STDERR Redirection Test Completed =====
divya@Divyasree:~/stderr_redirect_project$ cat no_error.txt
divya@Divyasree:~/stderr_redirect_project$ ls -l no_error.txt
-rw-r--r-- 1 divya divya 0 Sep 27 14:55 no_error.txt
divya@Divyasree:~/stderr_redirect_project$ ./stderr_redirect
===== STDERR Redirection Manager =====

Enter command using stderr redirection:
Example: ls /missing 2> errors.txt
> ls
Error: stderr redirection '2>' not found.
divya@Divyasree:~/stderr_redirect_project$ |

```

Code: